

# Design of ALU Components. Addition and Subtraction

## 1 Purpose

The focus of the first two laboratory works is to show how the components (adders, multipliers, divisors) of the Arithmetic Logic Unit (ALU) can be designed and to highlight how different implementations can affect the performance and the usage of hardware resources.

## 2 Introduction

The Arithmetic and Logic Unit (ALU) is a digital circuit that performs arithmetic and logical components. This circuit is the basic block of the CPU and GPU and even for the simplest microprocessors.

An ALU must be able to calculate most operations. If there are more complex operations, the value of ALU is higher (i.e. more expensive), uses more space in the processor and dissipates more power. Thus, the design is a trade-off between computational power and area overhead/power consumption.

Almost all processor operations are done by one or more ALUs. An ALU loads the data from the input registers. The external control unit tells the ALU which operation (i.e. operation code) to perform on the data, and then the ALU stores the result into an output register. The control unit is responsible for moving the processed data between these registers, ALU and memory. Many of the ALU designs also contain a status register that indicates different cases, such as: carry-in or carry-out, overflow, divide-by-zero and so on.

## 3 Addition

The addition operation is the most frequently used arithmetic operation in any computer system. If there are more complex arithmetic functions, they are reduced to a series of additions. Note that by increasing the speed of addition, the speed of the ALU is also increased; but the speed and cost of adders are directly proportional to their complexity.

### 3.1 Full Adder

This circuit is the basic addition block, which adds three 1-bit inputs: 2 bits to be added ( $x_i$  and  $y_i$ ) and a 1 bit carry-in ( $C_i$ ). It generates 2 outputs: the sum bit ( $S_i$ ) and the carry-out ( $C_{i+1}$ ) bit. The figure below illustrates a full adder symbol.

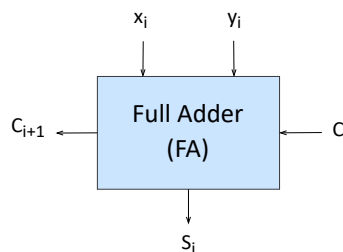


Figure 1: Full adder

| $x_i$ | $y_i$ | $C_i$ | $S_i$ | $C_{i+1}$ |
|-------|-------|-------|-------|-----------|
| 0     | 0     | 0     | 0     | 0         |
| 0     | 0     | 1     | 1     | 0         |
| 0     | 1     | 0     | 1     | 0         |
| 0     | 1     | 1     | 0     | 1         |
| 1     | 0     | 0     | 1     | 0         |
| 1     | 0     | 1     | 0     | 1         |
| 1     | 1     | 0     | 0     | 1         |
| 1     | 1     | 1     | 1     | 1         |

Table 1: Full adder truth table

Given the truth table above, the Boolean expressions of the outputs (after reduction) are:

$$S_i = x_i \oplus y_i \oplus C_i$$

$$C_{i+1} = x_i \cdot y_i + (x_i + y_i) \cdot C_i$$

A **half adder** is basically a full adder without the carry input and it generates a sum bit and a carry bit.

### 3.2 Ripple-Carry Adder

This type of adders are implemented by several full adders in series, each carry output is connected to the input of the next one. This is a parallel adder and is used for adding n-bit number. It has the advantage of simplicity and low cost, but at the cost of reduced speed. Figure 2 illustrates the block diagram of a ripple carry adder for adding 4-bit numbers.

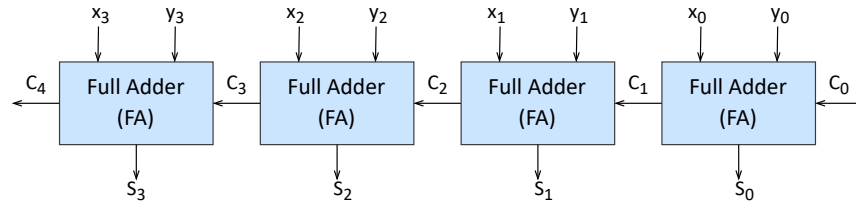


Figure 2: 4-bit ripple-carry adder using 4 full adders

### 3.3 Carry Lookahead Adder

In order to reduce the time required to form the carry signals, this type of adders use a separate block which calculates the carry output of each full adder. Thus, there is no “wait time” for carries to ripple from stage to stage (like in the ripple carry adder design). In the figure below, the block diagram of a carry lookahead adder that adds 4-bit integers is exposed.

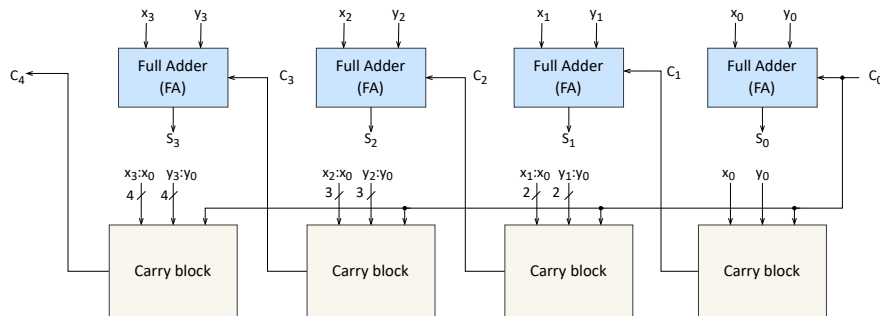


Figure 3: Carry lookahead adder design for adding 4-bit numbers

Given the design above, two functions are used to *generate* ( $g$ ) and *propagate* ( $p$ ) the carry output.

$$g_i = x_i \cdot y_i$$

$$p_i = x_i + y_i$$

Using the 2 functions above, the output carry can be expressed as:

$$C_{i+1} = g_i + p_i \cdot C_i$$

Thus, each carry output can be formed using the  $g$  and  $p$  functions from the same stage and from the previous stages. The carry blocks from the picture above can be joined in a single carry lookahead generator and each carry output can be generated by a combinatorial circuit.

The main advantage of such a type of adder is that the output carries can be individually computed based on the input carry and the bits in the operand. Thus, the latency is significantly reduced, as the computation of the output carry at level  $i + 1$  does not require all the previously  $i$  carry outputs to be already computed, as it can be expressed only in terms of the inputs. An example for two 4-bit operands,  $x(3..0)$  and  $y(3..0)$ , is given below:

$$C_1 = g_0 + p_0 \cdot C_0 = x_0 \cdot y_0 + (x_0 + y_0) \cdot C_0$$

$$C_2 = g_1 + p_1 \cdot C_1 = x_1 \cdot y_1 + (x_1 + y_1) \cdot C_1 = x_1 \cdot y_1 + (x_1 + y_1) \cdot (x_0 \cdot y_0 + (x_0 + y_0) \cdot C_0)$$

$$\begin{aligned} C_3 &= g_2 + p_2 \cdot C_2 = x_2 \cdot y_2 + (x_2 + y_2) \cdot C_2 = x_2 \cdot y_2 + (x_2 + y_2) \cdot (x_1 \cdot y_1 + (x_1 + y_1) \cdot C_1) \\ &= x_2 \cdot y_2 + (x_2 + y_2) \cdot (x_1 \cdot y_1 + (x_1 + y_1) \cdot (x_0 \cdot y_0 + (x_0 + y_0) \cdot C_0)) \end{aligned}$$

$$\begin{aligned} C_4 &= g_3 + p_3 \cdot C_3 = x_3 \cdot y_3 + (x_3 + y_3) \cdot C_3 = x_3 \cdot y_3 + (x_3 + y_3) \cdot (x_2 \cdot y_2 + (x_2 + y_2) \cdot C_2) \\ &= x_3 \cdot y_3 + (x_3 + y_3) \cdot (x_2 \cdot y_2 + (x_2 + y_2) \cdot (x_1 \cdot y_1 + (x_1 + y_1) \cdot C_1)) \\ &= x_3 \cdot y_3 + (x_3 + y_3) \cdot (x_2 \cdot y_2 + (x_2 + y_2) \cdot (x_1 \cdot y_1 + (x_1 + y_1) \cdot (x_0 \cdot y_0 + (x_0 + y_0) \cdot C_0))) \end{aligned}$$

### 3.4 Carry-Save Adder

This type of adder is used when more than 2 numbers are to be added (e.g., the partial products in a multiplier), because it reduces the carry propagation time. It adds three  $n$ -bit numbers  $x$ ,  $y$  and  $z$  using  $n$  independent full adders: the full adder  $i$  adds the bits  $x_i$ ,  $y_i$  and  $z_i$  (the third operand takes the place of the input carry) and produces a sum bit  $s_i$  and a carry bit  $c_i$ . Since the carries are not propagated between the full adders, all of them work in parallel, reducing the three numbers to two: an  $n$ -bit sum word  $s$  and an  $n$ -bit carry word  $c$ . To get the final result, the two words are added using a ripple-carry adder. As  $c_i$  has the weight  $2^{i+1}$ , the carry word is shifted one position to the left:  $x + y + z = s + 2c$ , so  $S_0 = s_0$  and the result has  $n + 2$  bits. Figure 4 illustrates a 4-bit carry save adder.

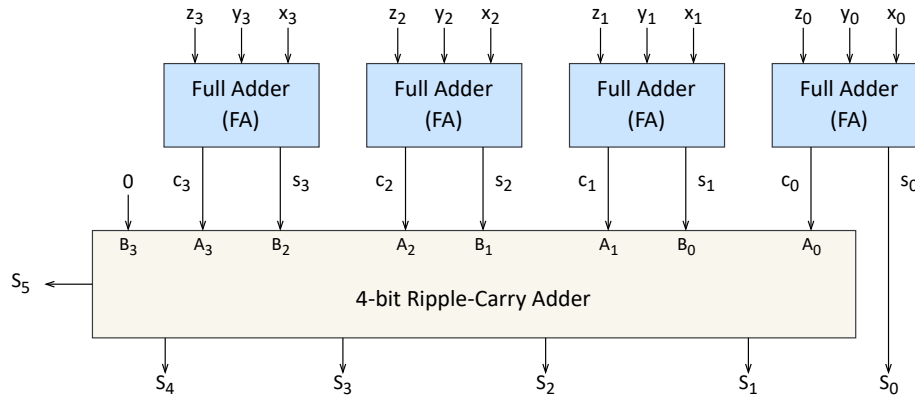


Figure 4: 4-bit carry-save adder

### 3.5 Carry-Select Adder

This type of adder uses redundant hardware to speed up the addition process. It divides the adding operation into 2 parts: the high-order half and the low-order half. First, the high-order half of the sum is calculated for both possible input carries. When the carry from the low-order half of the sum is known, the proper high-order half can be selected. Note that there is the possibility to divide the adder into quarters, so even lower complexity. This is why the complexity of this type of adder overcomes the carry lookahead adder disadvantages. Figure 5 illustrates such a 4-bit block: the sum is computed by two ripple-carry adders, one with the input carry 0 and one with the input carry 1, and the correct result is selected by multiplexers using  $C_{in}$  (the carry from the lower-order block).

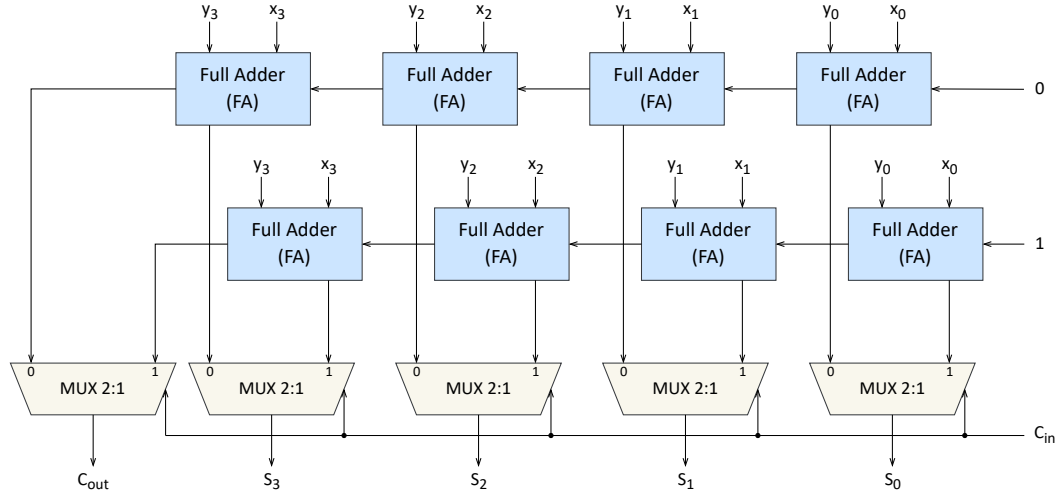


Figure 5: 4-bit carry-select adder block

### 3.6 Serial Adder

This is the simplest type of adder because it uses a single full adder and a D flip-flop. Basically, it performs the addition step by step from the Least Significant Bit (LSB) to the Most Significant Bit (MSB). The D flip-flop is used to propagate the carry of the previous sum to the next bits that are going to be added. As shown in Figure 6, the operands are stored in two shift registers, which provide one bit of each operand ( $x_i$  and  $y_i$ ) at every clock cycle, while the sum bit  $S_i$  is shifted back into register A, so that after  $n$  clock cycles register A holds the result.

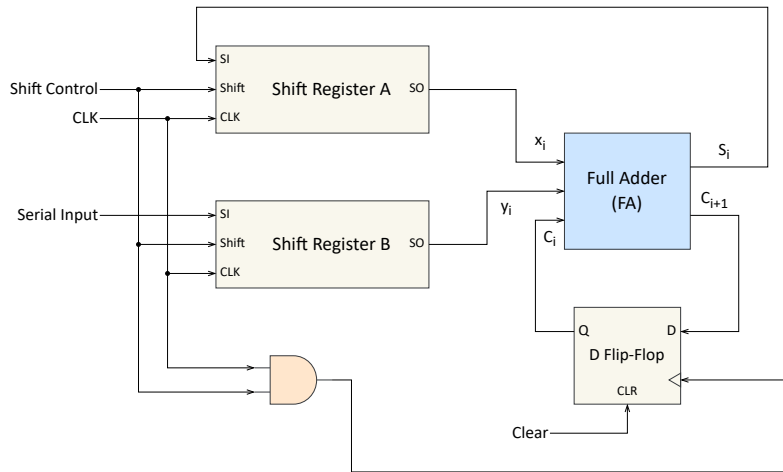


Figure 6: Serial adder

## 4 Subtraction

Subtraction does not require a dedicated circuit. By representing signed numbers in **two's complement (C2)**, the difference of two numbers can be computed as the sum between the first operand and the negated second operand, so the adders presented above can be reused:

$$A - B = A + (-B)$$

### 4.1 Two's Complement Representation

On  $n$  bits, a number in two's complement can take values in the range  $[-2^{n-1}, 2^{n-1} - 1]$  (e.g.,  $[-8, 7]$  for  $n = 4$ ). The Most Significant Bit (MSB) indicates the sign of the number (0 for positive, 1 for negative) and has the weight  $-2^{n-1}$ , while all the other bits have their usual positive weights:

$$X = -x_{n-1} \cdot 2^{n-1} + \sum_{i=0}^{n-2} x_i \cdot 2^i$$

The two's complement of a number  $B$  (i.e.,  $-B$ ) is defined as  $2^n - B$ . It can be obtained easily by inverting all the bits of  $B$  (one's complement,  $\overline{B}$ ) and adding 1:

$$-B = \overline{B} + 1$$

For example, on 4 bits,  $3 = 0011$ , so  $-3 = \overline{0011} + 1 = 1100 + 1 = 1101$ .

Therefore, the subtraction becomes:

$$A - B = A + \overline{B} + 1$$

### 4.2 Adder/Subtractor

The expression above can be implemented directly using an  $n$ -bit adder (ripple-carry, carry lookahead, etc.):

- the inversion of  $B$  is done using an XOR gate on each bit  $y_i$  of the second operand, having as second input a control signal  $Op$  (0 for addition, 1 for subtraction). Since  $y_i \oplus 0 = y_i$  and  $y_i \oplus 1 = \overline{y_i}$ , the adder receives either  $B$  or  $\overline{B}$ ;
- the  $+1$  is obtained for free by connecting the same control signal  $Op$  to the input carry  $C_0$  of the adder.

Figure 7 illustrates a 4-bit adder/subtractor built on top of the ripple-carry adder from Figure 2. The same XOR stage can be placed in front of any other adder (e.g., the carry lookahead adder).

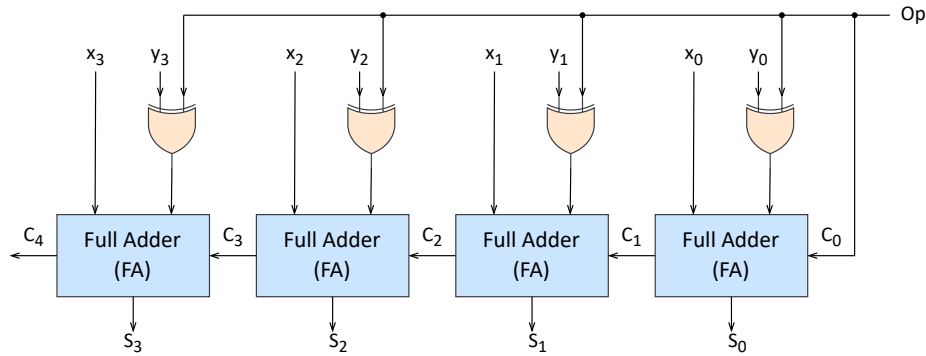


Figure 7: 4-bit adder/subtractor based on a ripple-carry adder

$$y'_i = y_i \oplus Op, \quad C_0 = Op$$

$$S = A + (B \oplus Op) + Op = \begin{cases} A + B, & Op = 0 \\ A + \overline{B} + 1 = A - B, & Op = 1 \end{cases}$$

Thus, the same hardware (an adder and  $n$  XOR gates) performs both addition and subtraction, and the result is correct for both signed (C2) and unsigned operands.

### 4.3 Carry and Overflow

When the operands are interpreted as **unsigned** numbers, the output carry  $C_n$  of a subtraction is the inverse of the borrow:  $C_n = 1$  means  $A \geq B$  (no borrow), while  $C_n = 0$  means  $A < B$  (borrow).

When the operands are interpreted as **signed** (C2) numbers, the result may fall outside the representable range. This situation is called *overflow* and is detected by comparing the carry in of the MSB with the carry out of the MSB:

$$V = C_n \oplus C_{n-1}$$

Equivalently, overflow occurs in a subtraction only when the operands have different signs and the sign of the result differs from the sign of  $A$ .

### 4.4 Examples

Table 2 shows three 4-bit subtractions performed as  $A + \overline{B} + 1$ .

| Operation      | $A$  | $\overline{B}$ | $C_0$ | $S$       | $C_4$ | $C_3$ | $V$ |
|----------------|------|----------------|-------|-----------|-------|-------|-----|
| $5 - 3 = 2$    | 0101 | 1100           | 1     | 0010 (2)  | 1     | 1     | 0   |
| $3 - 5 = -2$   | 0011 | 1010           | 1     | 1110 (-2) | 0     | 0     | 0   |
| $6 - (-3) = 9$ | 0110 | 0010           | 1     | 1001 (-7) | 0     | 1     | 1   |

Table 2: Examples of 4-bit subtraction in two's complement

In the first example, the result is correct and the carry  $C_4 = 1$  is simply discarded. In the second example the result 1110 is the C2 representation of  $-2$ . In the last example, the correct result (9) cannot be represented on 4 bits (maximum value is 7), so the adder returns  $-7$  and the overflow flag is set ( $V = 1$ ).

## 5 Exercises

1. Design and implement in VHDL a 4-bit carry lookahead adder.
2. Extend the 4-bit carry lookahead adder implemented in exercise 1 to support both addition and subtraction.
3. Using the 4-bit ripple-carry adder implemented in the previous laboratory, design and implement in VHDL 4-bit carry save adder.

## References

- [1] Ripple-Carry Adder [accessed Oct. 2026], <https://www.sciencedirect.com/topics/computer-science/ripple-carry-adder>
- [2] Carry Look-Ahead Adder [accessed Oct. 2026], <https://www.geeksforgeeks.org/carry-look-ahead-adder/>
- [3] Arithmetic logic unit [accessed Oct. 2026], [https://en.wikipedia.org/wiki/Arithmetic\\_logic\\_unit](https://en.wikipedia.org/wiki/Arithmetic_logic_unit)